SCOPE · LEARNING · COMPETITION · INDUSTRY

VANTA

Virtualized Adversarial Network & Technical Assessment

One engine that generates broken systems. Three things built on it: learn, compete, assess.

What VANTA's scope really is, how to make students learn from it, how to run Battlecode-, ICPC- and hackathon-style competitions with DSA and competitive programming inside, and how it could succeed as an industry product.

PRODUCT MODES	COMPETITION FORMATS	PROPOSED DEFECT CLASSES	ROADMAP PHASES
3	6	20	4

Contents

1 The answer on one page 3
- 1.1 The scope, in one sentence 3
- 1.2 What to do, in order 3
- 1.3 Why this is worth doing now 3

2 Clarifying the scope 5
- 2.1 Why the scope feels unclear 5
- 2.2 What VANTA is today (honest inventory) 6
- 2.3 The one thing VANTA has that others don't 6
- 2.4 In scope and out of scope 6

3 Making students actually learn 8
- 3.1 Skills VANTA can teach 8
- 3.2 Learning-science principles to build on 8
- 3.3 The learning loop 9
- 3.4 Features to add, and the principle each one serves 9
- 3.5 Measuring whether students learn 11

4 Bringing in DSA and competitive programming 12
- 4.1 Why DSA fits VANTA naturally 12
- 4.2 Four new kinds of world 12
- 4.3 Kind A: broken data structures 12
- 4.4 Kind B: "it's correct but too slow" 13
- 4.5 Kind C: optimization worlds 14
- 4.6 Kind D: adversarial worlds (games) 14
- 4.7 What changes in the engine 14

5 Competition formats 15
- 5.1 Sprint (ICPC-style): the first format to build 15
- 5.2 Marathon (heuristic optimization) 16
- 5.3 Arena (Battlecode-style): the flagship 16
- 5.4 Hackathon (24–48 hours, teams) 17
- 5.5 Incident Night (CTF-style board) 17
- 5.6 League (a semester at a college) 18
- 5.7 Integrity in the age of AI 18

6 Running the first event: VANTA Sprint #1 19
- 6.1 The event at a glance 19
- 6.2 Timeline 19
- 6.3 Infrastructure for about 80 people 19
- 6.4 Roles 20
- 6.5 What to measure at the first event 20

7 Roadmap 21
- 7.1 Phase 1: "It teaches, and it can host a contest" (6–8 weeks) 21
- 7.2 Phase 2: "A season people look forward to" (next semester) 21
- 7.3 Phase 3: "Something others pay for or depend on" (year 2) 22
- 7.4 Building order inside each phase 22

8 Making VANTA successful in industry 23
8.1 Four markets VANTA can serve 23
8.2 Where VANTA sits 24
8.3 The competitive landscape 24
8.4 Business models that fit 25
8.5 Going to market: start where you are 25
8.6 What makes it defensible 25
8.7 Metrics that matter 26
8.8 What a professor, a company, or an investor will want to see 26

9 Risks and how to handle them 27

10 Your next steps 28
10.1 How to present this to your professor 28

A Proposed archetype catalogue 29
A.1 Catalogue 29
A.2 What each archetype must include 30

B Scoring formulas 31
B.1 Sprint (ICPC rules) 31
B.2 Marathon (relative scoring) 31
B.3 Incident Night (dynamic points) 31
B.4 Arena (ratings and matches) 31
B.5 Hackathon (rubric) 32
B.6 Practice points (Learn mode) 32

C Arena game sketch, "Contention" 33
C.1 The world 33
C.2 A turn 33
C.3 Scoring a match 33
C.4 The baseline bot and its planted bugs 33
C.5 The DSA a strong bot needs 34
C.6 A suggested season calendar (4 weeks) 34

D Sources 35
D.1 Competitions and formats 35
D.2 AI and integrity 35
D.3 Learning science 35
D.4 Markets and employability 36
D.5 VANTA itself 36

CHAPTER 1

The answer on one page

1.1 The scope, in one sentence

> ***VANTA is an engine that generates realistic, broken-but-fixable software systems on demand and proves them by running them. Everything else is a mode built on that engine:*** *practice, courses, timed contests, Battlecode-style arenas, hackathons and hiring assessments.*

This is why the scope has felt unclear. Every new idea seemed to be a different project. They are really the same engine with a different wrapper. Once that is clear, the scope becomes three products that share one core:

Product	For whom	What it is	Why VANTA wins here
VANTA Learn	Students, instructors	Tracks of generated worlds, with hints, debriefs and spaced review, tied to course topics (C, debugging, DSA, performance)	Every student gets a unique world for the same learning goal, so there is no answer key to copy
VANTA Compete	Clubs, colleges, hackathon organizers	Six contest formats, from a 2-hour ICPC-style Sprint to a month-long Battlecode-style Arena	Problems are generated per event and per person, proven solvable, and admit several correct designs
VANTA Assess	Companies, placement cells	Realistic "fix this system" assessments with process evidence (experiments run, witnesses asked)	Leak-resistant by construction, and closer to real engineering work than puzzle questions

1.2 What to do, in order

1. **Make it teach.** Add a *debrief* after every solve or give-up: what was wrong, the evidence that pointed to it, and the other correct approaches. Add a *hint ladder* and *tracks*. Without these, VANTA tests skills but doesn't build them (Chapter 3).
2. **Add DSA and performance worlds.** Add new archetypes where the bug is a broken data structure, or where the code is correct but too slow for the hidden large inputs. The evolution loop already proves "broken but fixable"; it extends naturally to "too slow but speedable" (Chapter 4).
3. **Run one contest format well before building six.** Start with **VANTA Sprint**: a 2–3 hour, ICPC-style contest of 5–6 generated worlds, with a scoreboard (Chapter 5). Then build the Battlecode-style **Arena** as the flagship for the next semester.
4. **Pilot at your own university and measure.** Run a pre-test and post-test with fresh seeds, collect solve data and surveys, and write it up. This data, more than features, is what will convince a professor, a department, or a company (Chapters 6–7).
5. **Then go to industry with evidence.** Start with universities and competitions, which are cheap to reach and produce data. Approach assessments once you can show validity and integrity (Chapter 8).

1.3 Why this is worth doing now

- **The usual problem banks are breaking.** Assessment vendors now run automated leak detection and swap out questions found online. HackerRank, for example, scans the web for its library questions *daily* and

replaces leaked ones automatically. Generated worlds remove that problem at the source.

- **AI has changed competitive programming.** Google DeepMind reports that Gemini solved 10 of 12 problems at the 2025 ICPC World Finals, which would have placed it 2nd among university teams. Puzzle-style problems are increasingly solvable by AI. Work that needs investigation, judgement between trade-offs, and an explanation is where human skill still shows.
- **There is a gap in big public competitions.** Google closed Code Jam, Hash Code and Kick Start in February 2023. Together they had engaged more than 300,000 engineers a year. Battlecode shows the appetite for long-running strategy competitions; it has run at MIT since 2007.
- **Employability is a live problem in India.** The Mercer-Mettl India Graduate Skill Index 2025 puts overall graduate employability at 42.6%. A tool that builds and proves practical engineering skill has an obvious audience.

The honest bottom line

VANTA's engine is genuinely novel, but today it is a strong prototype with two defect classes. Its success depends on three things, in this order: a growing **library of archetypes**, a **learning loop** that measurably improves students, and **one competition format** run so well that people ask for the next one. Everything in this document serves those three.

CHAPTER 2

Clarifying the scope

2.1 Why the scope feels unclear

Over its development, VANTA has been described as several things: a hackathon platform, a debugging trainer, a Discord game, an assessment tool, an evolutionary AI system, and now a Battlecode-like competition. Each description is true. They feel like different projects because they describe *what people do with VANTA*, not *what VANTA is*.

The fix is to separate four layers, so each idea has an obvious place:

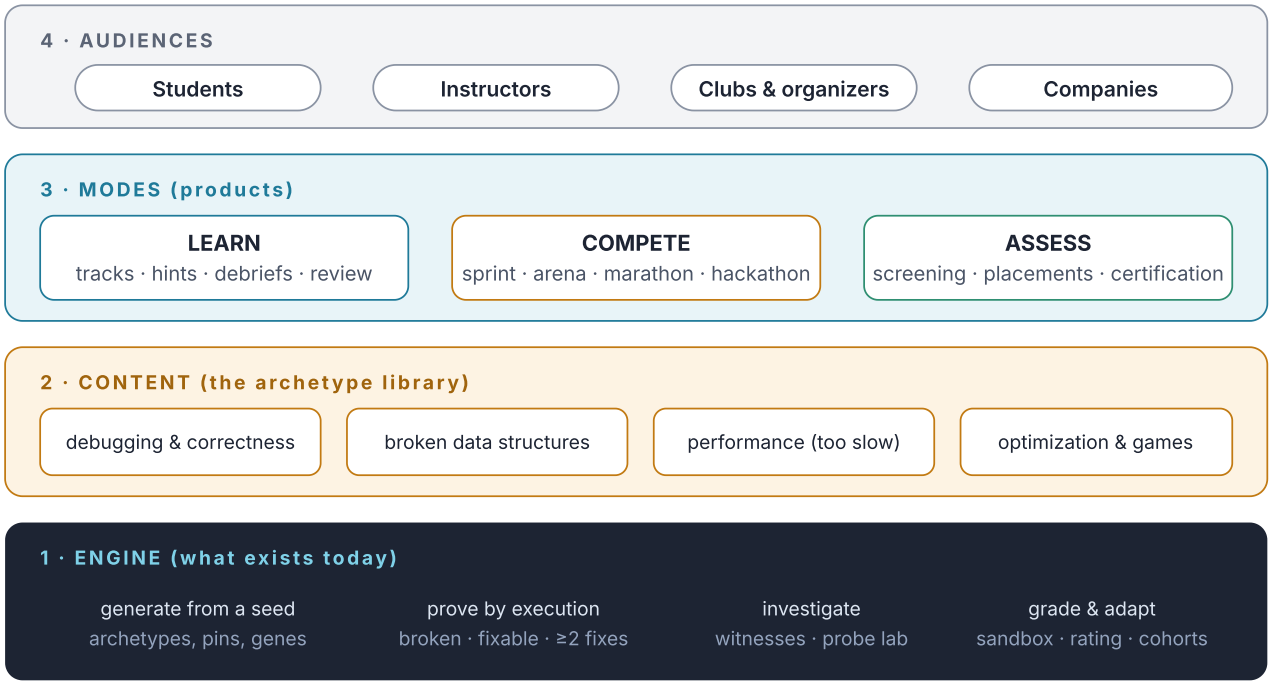


Figure 2.1. The scope map. Layer 1 is built. Layer 2 is the main growth area. Layer 3 is where most of your ideas belong. Layer 4 is who pays attention, and later who pays.

- **Engine** (built). Generates a world from a seed, proves it by compiling and running it, lets people investigate it, and grades and adapts. Every mode reuses it unchanged.
- **Content** (the main growth area). The *archetype library*. Each new archetype adds a whole family of problems to every mode at once. This is VANTA's equivalent of a problem bank, except that each entry is a generator rather than a fixed question.
- **Modes** (where your ideas go). Learn, Compete and Assess are three ways of packaging worlds: different rules, timing, scoring and screens, on the same engine.
- **Audiences.** Who uses each mode. Students and clubs come first because they are reachable and generate data. Instructors come next because they bring whole classes. Companies come last because they need evidence before they pay.

2.2 What VANTA is today (honest inventory)

Area	Built and tested	Not built yet
Engine	Generation from seeds; evolution proving viability; C sandbox with sanitizers; grading; Elo-style rating; cohorts with unique identifiers; originality analysis	Container-grade sandbox; compute budgets; non-C languages
Content	2 archetypes (scheduler fairness, resource pool), 3 bug variants each	DSA archetypes, performance archetypes, optimization and game archetypes
Learn	Witnesses, probe lab, replay, calibrated difficulty, in-app guide	Debriefs, hints, tracks, spaced review, instructor console
Compete	Cohorts (same defect, unique worlds) and a host console	Timed contests, scoreboard, penalties, divisions, arena, team play
Assess	Contestant-safe API, hidden checks, admin ground truth	Proctoring options, reports for recruiters, validity studies
Interfaces	Web workbench, Discord bot (16 commands)	Replay viewer for games, mobile layout polish

2.3 The one thing VANTA has that others don't

Most platforms share the same basic design: a *bank of hand-written problems* plus a grader. VANTA's design is different in five ways that matter:

1. **Generated, not stored.** There is no answer key to leak, sell or memorize. Two people can face the same problem without being able to share code.
2. **Proven before served.** Every world is compiled and run to show it is broken, fixable, and fixable in at least two different ways.
3. **Many right answers.** This makes VANTA suitable for design and trade-off thinking, not just "find the one trick".
4. **Investigation, not interrogation.** Evidence comes from running the system (witnesses and experiments), so learners practise the real debugging loop: hypothesis → experiment → conclusion.
5. **Adaptive by construction.** The same generator serves a first-year and a final-year student, because difficulty is a dial on the generator.

The positioning that follows from this:

> *LeetCode and Codeforces train you to write an algorithm from a blank page. VANTA trains you to take a real, broken system and make it correct, fast and fair. That is most of what software engineering jobs actually are.*

2.4 In scope and out of scope

Saying "no" is how a student project stays finishable.

In scope (do it)	Out of scope for now (don't)
C first, then C++ and Python once the archetype kit is stable	A general online IDE or a new programming language

In scope (do it)	Out of scope for now (don't)
Debugging, DSA correctness, performance and optimization worlds	Hand-written problem banks (this would throw away the advantage)
One contest format at a time, run well	Building all six formats before running one
Measuring learning with fresh seeds	Claiming learning gains without data
Universities, clubs and competitions first	Enterprise hiring sales before you have validity evidence
AI policy as a rule set (allowed or not allowed, logged)	Promising that AI can never solve VANTA worlds (it can)
An archetype authoring kit, so others can add content	You personally writing every archetype forever

CHAPTER 3

Making students actually learn

Today VANTA is good at *testing* whether someone can fix a system. To *build* the skill, it needs a learning loop around each world. This chapter covers which skills VANTA can teach, which learning-science principles to use, and the concrete features that follow from them.

3.1 Skills VANTA can teach

Skill	Why it matters	How VANTA exercises it
Debugging as a method	Debugging is a core skill that is rarely taught explicitly. McCauley et al. (2008) reviewed the literature on this	Witnesses and experiments force hypothesis → test → conclusion instead of random edits
Reading unfamiliar code	Most professional work starts in someone else's code	Every world is new code with new names
Designing test cases	Edge cases are where bugs live	The probe lab is test design. Hidden checks reward thinking about empty input, exact fits and repeats
C fundamentals	Pointers, arrays, integer division, undefined behaviour	Sanitizers turn memory errors into visible failures; variants like integer-division scoring target misconceptions
DSA selection	Knowing *which* structure fits	DSA and performance worlds (Chapter 4) where the fix is a better structure
Complexity analysis	Correct but too slow is still wrong	Performance worlds with large hidden inputs and measured time limits
Trade-off reasoning	Real fixes help some users and hurt others	Witness replays show who wins and who loses under each correct design
Explaining a decision	Engineers must justify changes	"Explain your fix" write-ups, peer-reviewed in cohorts

3.2 Learning-science principles to build on

- **Productive failure.** Letting learners struggle with a problem *before* instruction can improve conceptual understanding and transfer. A meta-analysis by Sinha and Kapur (2021, *Review of Educational Research*) covered 53 studies and 166 comparisons, and found it works better for secondary and university students than for younger ones. VANTA is naturally "problem first". The missing piece is the *instruction after*: the debrief.
- **Deliberate practice at the edge of ability.** Practice is most effective when it is just beyond current skill, with immediate feedback. VANTA's calibrated difficulty already aims each world slightly above a person's rating.
- **Retrieval and spacing.** Coming back to a skill after a delay, without notes, strengthens it. VANTA can do this without repeating questions, because a new seed gives a new world for the same defect class.
- **Worked examples for beginners, fading as they improve.** Novices learn more from studying a solved example than from struggling blindly. Support should then fade as skill grows.

- **Feedback that explains, not just judges.** "Rejected: hidden check 3" tells a learner *that* they are wrong, not *why*. The debrief and hint ladder close that gap without giving the answer away up front.

3.3 The learning loop

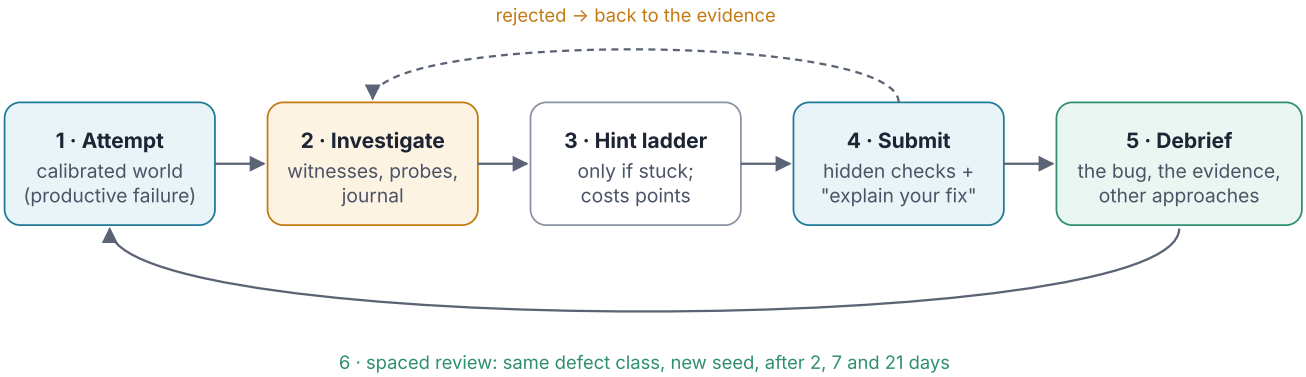


Figure 3.1. The learning loop. Steps 1, 2 and 4 exist today; 3, 5 and 6 are the additions that turn VANTA from a test into a teacher.

3.4 Features to add, and the principle each one serves

The debrief (most important)

After an accepted fix, or after the learner chooses "I give up, show me", show:

1. **What was planted**, in plain words, for example "release matched a stale holder id on an already-free slot". The host console already has these descriptions.
2. **The evidence trail.** Which witness answer or experiment would have revealed it, with the record attached. This teaches *where to look* next time.
3. **The approach gallery.** The reference designs, plus other learners' accepted solutions from different approach groups (the originality analysis already clusters them), with a sentence on the trade-off each one makes.
4. **One check question** to confirm understanding, for example: "Which requirement would pure shortest-first violate on a saturated queue?"

Principle: productive failure needs instruction *after* the struggle.

The hint ladder

Hints the learner unlocks one at a time, each costing practice points or contest score:

Rung	Hint	How VANTA computes it
1	Which **requirement** your code still breaks	Map each hidden check to the requirement it targets (archetype metadata)
2	Which **witness** saw the symptom	Run the witnesses on the learner's code and pick one whose answer differs from a reference fix's
3	A **minimal experiment** that reproduces it	Shrink the failing hidden scenario into a probe scenario
4	The **region of code** involved	The variant's known location in the starter

Principle: feedback that explains, faded support.

Tracks and mastery

Group defect classes into **tracks** with an order, for example:

Track	Contents	Unlock rule (example)
C Foundations	Off-by-one, integer division, array bounds, sanitizer errors	Start here
Debugging Method	Scheduler and pool worlds; witness and probe skills	3 solves in C Foundations
DSA in the Wild	Broken heaps, hash tables, binary search, BFS, union-find	2 solves at difficulty ≥ 0.4 in Debugging
Performance	Correct-but-slow worlds needing better complexity	3 DSA solves
Systems & Trade-offs	Multi-subsystem worlds, optimization worlds	Track badges from all of the above

Mastery means solving a defect class at or above a difficulty threshold *on more than one seed*, not solving one world once.

Principle: deliberate practice; mastery before moving on.

Spaced review

When someone fails or struggles with a defect class, bring it back later as a *new world* (new seed) after about 2, 7 and 21 days. They can't remember the answer, because the answer is different. They can only remember the *method*.

Principle: retrieval and spacing.

The investigation journal

A side panel where the learner writes one-line hypotheses. Each experiment or witness question is attached to the hypothesis it tests. On submit, the journal becomes part of the debrief and, in cohorts, part of what the host sees.

Principle: making thinking visible, so instructors can teach the method, not just the answer.

"Explain your fix"

After an accepted submission, ask for a 3–5 sentence explanation: what was wrong, why the change fixes it, and what it costs. In a cohort, each learner reviews two explanations from *different* approach groups. This turns the multi-solution property into peer learning.

Instructor console

This extends the cohort host console:

- a skill heatmap (students × defect classes);
- the most common wrong submissions per variant, which shows the misconceptions;
- the time-to-solve distribution and hint usage;
- students who are stuck right now, for live help in a lab.

Beginner mode

For the first two or three worlds:

- a worked example world, with a narrated walkthrough of an investigation;
- smaller code;
- witnesses whose first answers point more directly at the symptom.

This uses the `noise` and `reasoning_depth` genes, which the generator already carries but the current archetypes don't yet read.

3.5 Measuring whether students learn

Claims about learning need evidence. VANTA can collect it cheaply, because it can generate equivalent but different tests on demand.

Measure	How
Learning gain	Pre-test and post-test on *the same defect classes with fresh seeds*: solve rate, time, hints used
Transfer	Post-test on a defect class not practised, but needing the same skill
Retention	Re-test 3–4 weeks later with new seeds
Method quality	From the journal: hypotheses tested before the first submission; experiments per solve
What helps	A/B test: half the cohort gets debriefs or hints, half doesn't; compare post-test results
Experience	A 5-question survey after each session (difficulty, usefulness, would-recommend)

A first study you can run this semester

Recruit 20–40 first-year students. **Week 1:** a 40-minute pre-test (2 worlds). **Weeks 2–4:** three 45-minute practice sessions with debriefs on. **Week 5:** a post-test with new seeds of the same classes, plus one transfer world. Report the solve-rate change, the median time-to-solve, and survey results. Even a small study like this is far more convincing to a professor than a feature list.

CHAPTER 4

Bringing in DSA and competitive programming

4.1 Why DSA fits VANTA naturally

Competitive programming tests whether you can find an algorithm that is *correct* and *fast enough*. VANTA's engine already proves "the starter is wrong, and a correct fix exists". It needs only one more measurement, time or operation count, to prove "the starter is too slow, and a fast-enough fix exists". The other properties carry over unchanged: generation from seeds, unique worlds per person, and at least two distinct reference fixes. For DSA, distinct fixes naturally means *different algorithms*: a heap versus a balanced tree, hashing versus sorting, BFS versus union-find.

4.2 Four new kinds of world

Kind	The starter is...	The contestant must...	Competitive-programming analogue
A. Broken data structure	A data structure with a planted bug, used by a realistic system	Find and fix the invariant violation	Implementation problems, debugging rounds
B. Too slow	Correct, but with the wrong complexity for the hidden input sizes	Choose a better structure or algorithm	Classic ICPC/Codeforces problems with time limits
C. Optimization	A valid but poor solution to a hard (often NP-hard) problem	Improve a score; there is no single optimum	AtCoder Heuristic Contest, the former Google Hash Code
D. Adversarial	A baseline bot in a generated game	Write a strategy that beats other people's code	Battlecode, CodinGame bot arenas

Kinds A and B fit the current engine almost directly. Kind C needs score-based grading. Kind D needs a game engine and a match runner (Chapter 5).

4.3 Kind A: broken data structures

Each one is a realistic system with a data structure at its core and a planted invariant violation. Some examples (Appendix A has the full catalogue):

System	Structure	Planted variants	Distinct correct fixes
Hospital triage queue	Binary heap	sift-down compares with the wrong child; heap size not decremented; parent index off by one	array heap; pairing via sorted insertion (if the size allows); bucket queue for small priority ranges
Username registry	Open-addressing hash table	deletion breaks the probe chain (no tombstones); load factor never triggers a resize; hash uses a signed `%`	tombstones; backward-shift deletion; separate chaining
Course prerequisite checker	Graph traversal	BFS marks visited on pop, not push; a cycle is reported only from node 0	Kahn's algorithm; DFS with three colours

System	Structure	Planted variants	Distinct correct fixes
Network outage regions	Union-find	union by the wrong root; `find` without returning the compressed root	union by rank; union by size; path halving
Seat finder	Binary search	`lo <= hi` with `mid = (lo + hi)/2` overflow; returns any match instead of the first	lower-bound style; half-open interval style

4.4 Kind B: "it's correct but too slow"

This is the new idea that brings competitive programming directly into VANTA.

How it works. The starter passes all *small* hidden tests and gives correct answers. Hidden *large* tests exceed the time limit. For example, a log analyser that finds duplicate session ids with nested loops ($O(n^2)$). It is correct on 1,000 records and hopeless on 2,000,000.

How the engine proves it.

1. Evolution chooses the input size *N* from the `constant_magnitude` gene, so difficulty also means "how much better than the naive complexity you need to be".
2. The harness runs each reference fix on the large tests and measures it. The **time limit** is set from the measurement, for example 3× the slowest reference fix, rather than guessed.
3. The world is **viable** only if the starter *times out*, and at least two different efficient fixes (say, hash-based and sort-based) pass with margin.

Fair timing. Wall-clock time varies between machines and with load. Battlecode deals with this by counting computation: its Java engine "accurately tracks how much computation a competitor is using". VANTA can do the same in C in either of two ways:

- **Operation counters.** The harness provides the data-access functions and counts calls. This is exact and fully deterministic, but it constrains how contestants write code.
- **CPU time with repetition.** Measure process CPU time over several runs, take the median, and use a generous multiple of the reference. This is simple and works for any code.

Start with the second, and add the first for Arena bots, where per-turn fairness matters.

Example performance worlds.

System	Naive starter	Faster designs (distinct fixes)
Duplicate session finder	$O(n^2)$ pair check	Hash set O(n); sort then scan O(n log n)
Top-k trending tags	Re-sort the whole list per query	Heap of size k; counting with bucket selection
Range booking checker	Scan every booking per query	Sort + sweep; interval tree; difference array (bounded ranges)
Route planner	BFS from scratch per query on a weighted graph	Dijkstra with a binary heap; precomputation per source
Rolling average service	Recompute the window sum each tick	Running sum; prefix sums
Friend-group counter	Re-run a flood fill per new friendship	Union-find; incremental BFS with merging

4.5 Kind C: optimization worlds

Some real problems have no perfect answer that can be found in time: scheduling with deadlines, packing, routing, assignment. AtCoder's Heuristic Contest (AHC) is built around these. Its statement describes the goal as creating "a better solution to a problem for which it is difficult to find the optimal solution". Rankings shown during the contest are provisional, and final standings use system tests on more inputs. Google's Hash Code used the same style.

A VANTA optimization world:

- **is generated** like any other world: a facility (labs, buses, servers) with stakeholders, constraints and a score, for example total weighted waiting time with a fairness penalty;
- **has a baseline** starter that is valid but poor (first-come-first-served, greedy);
- **is scored relatively** on hidden instances: your score on an instance divided by the best submitted score, summed across instances;
- **uses the witnesses as qualitative feedback**: who is unhappy with your schedule and why.

These worlds are where "many correct solutions" stops being a property and becomes the whole point. They are good for multi-day Marathon contests and for teams.

4.6 Kind D: adversarial worlds (games)

Chapter 5 describes the Battlecode-style Arena in full. In DSA terms, good bots need:

- pathfinding (BFS, A*);
- priority queues for scheduling actions;
- matching and assignment;
- caching under memory limits;
- careful complexity under a per-turn compute budget.

Arena is where DSA stops being exercises and becomes strategy.

4.7 What changes in the engine

Change	Needed for	Effort
Measured time limits from reference runs; CPU-time median over repeats	Kind B	Small
Large-input generation inside the harness (from a seed, not stored as files)	Kind B	Small
Score-based grading (`VANTA_SCORE|instance|value` lines) and relative scoring	Kind C	Medium
Viability rules per kind ("starter times out", "baseline score < reference score by X%")	B, C	Small
Deterministic game engine, match runner, replay format, compute budget	Kind D	Large
Archetype authoring kit: template, checklist, test helpers, "viability report"	All	Medium, and the most valuable long-term

CHAPTER 5

Competition formats

VANTA can run six kinds of competition on one engine. They differ in length, team size, and what "winning" means. Run them one at a time. Sprint first.

Format	Inspired by	Length	Teams	Worlds	Winner decided by
Sprint	ICPC, Codeforces rounds	2–3 hours	1–3	5–7 worlds of kinds A, B and debugging	Worlds solved, then penalty time
Marathon	AtCoder Heuristic Contest, Hash Code	1–10 days	1–4	1 optimization world (kind C)	Relative score on hidden instances
Arena	Battlecode, CodinGame bot programming	3–4 weeks	1–4	1 generated game (kind D)	Tournament results
Hackathon	Smart India Hackathon, company hackathons	24–48 hours	2–5	1 large multi-subsystem world	Hidden tests + stakeholder score + judges
Incident Night	Jeopardy-style CTFs, such as picoCTF	3–6 hours	1–4	Board of 15–30 worlds by category	Points; first-solve bonus
League	Codeforces divisions, CodinGame leagues	A semester	1	Weekly practice + monthly Sprints	Rating and division promotion

5.1 Sprint (ICPC-style): the first format to build

Format. Around 2.5 hours, 6 problems (A to F) of increasing difficulty, individual or teams of up to 3. Each *problem* is a defect class pinned for everyone (like a cohort). Each *participant* gets a unique world for it, with their own names and constants. A suggested mix:

Slot	Kind	Example	Target difficulty
A	Debugging (warm-up)	Pool leak, off-by-one variant	0.2
B	Broken data structure	Binary search returns any match, not the first	0.35
C	Debugging	Scheduler, no-close-check variant	0.45
D	Too slow	Duplicate session finder ($O(n^2)$ → hash or sort)	0.55
E	Broken data structure	Heap sift-down picks the wrong child	0.65
F	Too slow + trade-off	Top-k trending tags under memory limits	0.8

Scoring. Use ICPC rules, which are simple and well understood. Contestants are ranked by problems solved, then by total time. Total time is the sum, over solved problems, of the minutes from the start of the contest until the accepted run, plus a 20-minute penalty for every rejected run on that problem. (This is the published ICPC regional rule; CERC 2025's rules are cited in the sources.) Compile errors can be exempted from penalties, which helps beginners.

Divisions. Use VANTA ratings, like Codeforces divisions:

- **Div. 3** for ratings below 1300;
- **Div. 2** for ratings below 1600;

- **Div. 1** for everyone.

Run them on the same day, with problems shifted in difficulty.

Scoreboard. Show it live and freeze it for the final 30 minutes. Reveal the final standings at the closing session, which is the most exciting moment of an ICPC contest.

What must be built. A contest object holding the problem list (each problem a pinned cohort), start and end times, and penalty rules. Add a scoreboard page, a contest view for contestants (problem tabs, timer, per-problem verdicts), and team accounts.

5.2 Marathon (heuristic optimization)

One optimization world (Chapter 4, kind C), for example *"Exam Hall Scheduler: assign 4,000 exam sittings to rooms and slots. Minimize student clashes and walking distance; no room may be over capacity."*

- **Provisional and final scoring**, as in AtCoder Heuristic Contest. During the contest, submissions are scored on a public set of instances. Final standings use a separate hidden set, which discourages overfitting.
- **Relative scoring.** Per instance, score = your result ÷ the best result submitted. The total is the sum over instances, scaled to 10^9 points or similar.
- **A minimum interval between submissions**, for example 15–30 minutes. This keeps compute fair and encourages local testing. AHC enforces a 30-minute interval.
- **Witnesses as qualitative feedback**: "My exams were scheduled back to back in buildings 2 km apart."

A Marathon is approachable for beginners, because any valid answer scores something. Experts can keep climbing for days.

5.3 Arena (Battlecode-style): the flagship

Battlecode, run at MIT since 2007, gives teams game rules in early January. Teams refine an autonomous bot all month, play scrimmages, and compete in a single-elimination Sprint tournament and double-elimination qualifier tournaments before finals at MIT. Its organizers stress that the game changes every year, so strategies have to change too. CodinGame's bot arenas add *leagues* (Wood → Bronze → Silver → Gold → Legend). Each league has a boss bot you must beat to be promoted, and players are ranked within their league with TrueSkill.

VANTA Arena combines these with VANTA's own strengths:

- **A new generated game every season.** The game *family* is fixed by an archetype; maps and parameters come from seeds. Example family: **"Contention"**. Two to four teams' bots control couriers on a grid city and compete for shared resources: charging docks, delivery slots and narrow roads. Requests arrive over time, with deadlines and priorities.
- **Bots in C** (later C++ and Python) implement one function that is called every turn. Each turn runs under a **compute budget**: operations or instructions are counted, as Battlecode does by tracking computation, so efficient algorithms matter.

```
void act(const struct View *view, struct Orders *out);   /* called once per turn */
```

- **A baseline bot with planted bugs.** Every team starts from a provided bot that has a few VANTA-style defects: a pathfinder that revisits nodes, a scheduler that starves a far-away request, a resource counter that leaks. Week 1 of the Arena becomes a debugging challenge, and finding the bugs early is a real competitive advantage. This is unique to VANTA.

- **A ranked ladder.** Scrimmages run continuously. Ratings use TrueSkill or Glicko. Leagues have boss bots: the baseline, then an improved baseline, then last season's winner.
- **Tournaments.** A Sprint tournament after week 1 (single elimination), qualifiers after week 3 (double elimination), and a live final with replays on a big screen.
- **Replays and post-mortems.** Every match produces a deterministic replay (same seed, same bots, same result). After the season, the top teams present their strategies.

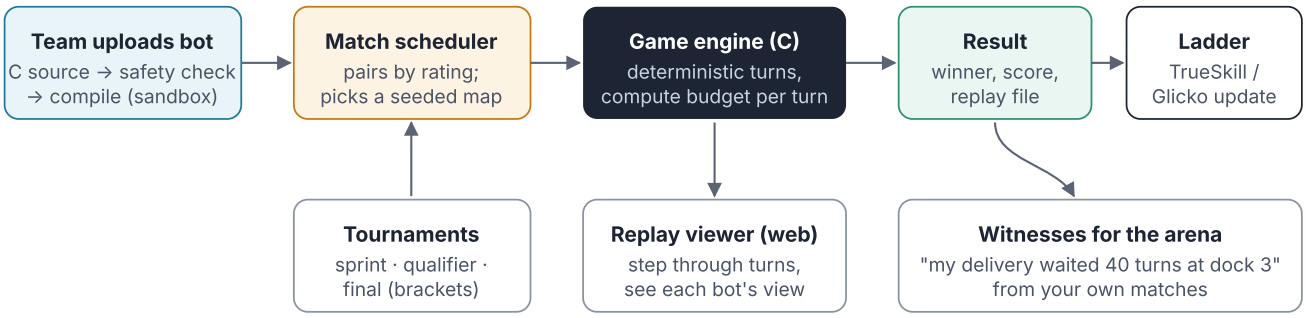


Figure 5.1. Arena architecture. The game engine and match runner are the largest new pieces; the sandbox, ratings and witnesses are reused.

5.4 Hackathon (24–48 hours, teams)

Each team gets a **large generated system**: several interacting subsystems (for example a campus facilities platform with booking, a laundry scheduler and a locker pool). It has 4–8 planted defects of different kinds, plus 2–3 **feature requests** from stakeholders that conflict with each other.

Component	Weight	How it's measured
Correctness	50%	Hidden checks across all subsystems (automatic)
Stakeholder satisfaction	20%	Witnesses relive the busy period under the team's code; each scores their experience (automatic, from traces)
Performance	10%	Large hidden inputs (automatic)
Engineering quality	20%	Judges review a 3-minute demo plus the "explain your changes" write-up

Because several correct designs exist, judges see genuinely different solutions. The automated 80% makes judging fast and fair, even with many teams.

Smart India Hackathon shows the scale of the appetite. Its 2025 edition received 68,766 team idea submissions, and 1,360 teams with more than 8,000 students reached the grand finale. VANTA can offer organizers something that format lacks: an objective, automatically graded core that can't be pre-prepared.

5.5 Incident Night (CTF-style board)

A jeopardy board with columns by category (Memory, Scheduling, Data structures, Performance, Optimization) and rows of 100–500 points.

- **Dynamic scoring.** Each world's value drops as more teams solve it, so rare solves are worth more. Appendix B gives a formula.
- **First-solve bonus** ("first blood"), announced live.
- **Unique worlds per team**, so a solution posted in a group chat doesn't work for anyone else.

The CTF world shows how competition and learning reinforce each other. picoCTF (Carnegie Mellon) has over 600,000 users. Learners asked for the challenges to stay open after the competition, which led to the non-competitive picoGym. VANTA should plan the same: **every contest becomes a practice set afterwards**, with new seeds.

5.6 League (a semester at a college)

Cadence	Activity
Weekly	Practice sets in each track (Learn mode); a 45-minute lab session
Monthly	A Sprint, with divisions by rating
Each semester	One Arena season (4 weeks) or one Marathon
Each year	An inter-college Hackathon or Arena final

Ratings carry across everything. Students are promoted between divisions, like Codeforces divisions and CodinGame leagues. Promotion is visible and earned, and demotion is gentle. One option is to never demote within a season, as CodinGame does within leagues.

5.7 Integrity in the age of AI

This has to be designed in from the start, not added later.

The facts:

- Frontier AI now performs at gold-medal level in the hardest student contests. Gemini solved 10 of 12 ICPC World Finals 2025 problems, which would have placed it 2nd.
- Codeforces (rule revised September 2024) allows AI for translating statements and minor code completion. It **forbids** feeding problems to AI for solutions, and using AI to diagnose errors from verdicts.
- Industry write-ups report AI-assisted cheating in technical interviews and assessments rising sharply in 2025. The numbers vary by source and should be read with caution. Vendors respond with plagiarism models, browser monitoring and proctoring. HackerRank, for example, lets organizations either allow AI with monitoring or restrict it.

What VANTA should do:

1. **Publish one clear AI rule per event, in tiers.** - *Open League*: AI assistants allowed and logged. This reflects real work, and the scoring values investigation and explanation. - *Classic*: no AI for solutions; allowed for language help, as in Codeforces' rule. - *Proctored*: for assessments and finals.
2. **Lean on what generation already gives you.** No two people share code. Nothing leaks before the contest, because nothing exists until generation. The originality analysis groups suspiciously similar novel code.
3. **Score the process, not just the result**, where it matters: experiments run, hypotheses in the journal, the explanation write-up. Add a short **"defend your fix" interview** for finals.
4. **Use formats where AI helps least.** In Arena strategy, multi-day Marathon optimization, hackathon trade-offs, and live finals, sustained human judgement and iteration still decide the winner.
5. **Don't overclaim.** An AI *can* read a VANTA world and fix it. VANTA's integrity comes from rules, uniqueness and process evidence, not from being unsolvable.

CHAPTER 6

Running the first event: VANTA Sprint #1

A concrete playbook for a first contest at your own university. It is designed to be run by one or two students with a faculty sponsor and almost no budget.

6.1 The event at a glance

Item	Plan
Name	VANTA Sprint #1: "Fix the Campus"
Who	30–80 first- and second-year students; individual entries plus an optional Div. 1 for strong students
When	One weekday evening or a Saturday afternoon; 2.5 hours of contest, plus a 30-minute briefing and a 30-minute debrief
Problems	5 worlds (A–E): 2 debugging, 2 broken data structures, 1 too-slow. Each is pinned for everyone, with a unique world per person
Scoring	ICPC: worlds solved, then penalty time (+20 minutes per rejected run); scoreboard frozen for the last 30 minutes
AI rule	"Classic": no AI for solutions. Language help is allowed. Explained in the briefing
Prizes	Certificates, department or club recognition, a sponsor's goodies if available; the top 3 present their approaches
Afterwards	Problems reopen as practice with new seeds; results write-up shared with participants and faculty

6.2 Timeline

When	Task
T − 4 weeks	Agree date and room with the faculty sponsor; announce; open registration (a cohort join code works as registration)
T − 3 weeks	Build the missing pieces: contest timer, scoreboard, penalties (Chapter 7, Phase 1). Containerize the sandbox
T − 2 weeks	Choose and test the 5 defect classes; run a mock contest with 5 friends; fix the rough edges
T − 1 week	Load test: simulate 80 participants submitting at once; check generation and grading times
T − 2 days	Generate everyone's worlds in advance (warm the cache); print the AI rule and help sheet
Day	Briefing (rules, the guide, a 5-minute demo) → contest → freeze → reveal → debrief of each problem
T + 1 week	Send results, survey and practice links; write a 2-page report with the data

6.3 Infrastructure for about 80 people

The current single-process server is fine for a classroom demo. For a contest:

- **Pre-generate worlds.** Joining a pinned cohort triggers evolution. Do it for every registered participant the day before, so nobody waits at the start.

- **Separate the workers.** Run grading and probes on a small worker pool (separate processes or machines) with a queue. A submission then never blocks the website.
- **Sandbox in containers.** No network, a read-only filesystem except a temporary folder, and memory, CPU and process limits. Do this before any public event.
- **Budget the load.** Assume 3–5 submissions and 10–20 experiments per person per hour. A sanitized compile-and-run takes roughly 1–2 seconds. Eighty people is therefore a few submissions per second at peak, which a handful of CPU cores can handle with a queue.

6.4 Roles

Role	Responsibility
Contest director (you)	Problem selection, rules, the final call on disputes
Tech lead	Servers, workers, the scoreboard, monitoring during the contest
Helpdesk (1–2 people)	Answer clarification questions (through a form, so answers are logged and shared with everyone)
Faculty sponsor	Room, legitimacy, certificates, and the bridge to the department

6.5 What to measure at the first event

- Participants, completion (submitted at least once), and solves per problem.
- Median time-to-solve per problem, and the rejected-run count before acceptance.
- Experiments and witness questions per solve. Do investigators do better?
- Distinct approaches per problem (from the originality analysis).
- A 5-question survey: difficulty, fairness, learning, fun, would-come-again.
- Incidents: crashes, grading delays, disputes, integrity concerns.

These numbers become the first page of your pitch to the department, to the next college, and eventually to companies.

CHAPTER 7

Roadmap

Four phases. Each has a clear goal, a short build list, and an exit test that must pass before moving on. The timings assume one student working part-time, with help from one or two friends in later phases.



Figure 7.1. The roadmap. Each phase ends with an exit test, not a date.

7.1 Phase 1: "It teaches, and it can host a contest" (6–8 weeks)

Build	Why
Debrief screen (bug explained, evidence trail, approach gallery)	Turns attempts into learning
Hint ladder, rungs 1–2 (failing requirement; which witness)	Unblocks beginners without giving answers
2 broken-data-structure archetypes (binary search, binary heap)	Brings DSA in
1 too-slow archetype (duplicate finder: $O(n^2)$ → hash or sort) with measured time limits	Brings competitive programming in
Contest mode: problem list of pinned cohorts, timer, penalties, frozen scoreboard, team entries	Enables Sprint
Container sandbox + worker queue	Required before any public event
Pilot study (Chapter 3) + Sprint #1 (Chapter 6)	Produces evidence

Exit test: at least 30 students have used it. The post-test shows a solve-rate improvement. Sprint #1 ran without grading failures. At least 70% of participants would come again.

7.2 Phase 2: "A season people look forward to" (next semester)

Build	Why
Arena v1: one game family ("Contention"), deterministic engine, match runner, ladder, replay viewer, tournaments	The Battlecode-style flagship
Tracks and mastery, spaced review	Long-term learning
Instructor console (skill heatmap, common mistakes, stuck students)	Makes adoption in courses easy

Build	Why
4–6 more archetypes (hash table, union-find, BFS/topological sort, top-k, range queries)	Breadth for contests and courses
An inter-college Sprint with 2–3 nearby colleges	Growth beyond one campus

Exit test: an Arena season completes with 15 or more teams. At least one course or lab uses VANTA for graded work. An inter-college event is held.

7.3 Phase 3: "Something others pay for or depend on" (year 2)

Build	Why
Assess pilot with 1–2 companies or a placement cell: proctoring options, recruiter report, validity study	The industry entry point
C++ and Python worlds	Matches what placement tests and contests use
Archetype authoring kit + contribution process (review, viability report)	Content can scale beyond you
Optimization worlds + a Marathon	A broader competition calendar
A sustainable model: sponsorships, university licences, or grants (Chapter 8)	It keeps running after you graduate

Exit test: at least one paying or sponsoring partner, or a formal departmental adoption. Assessment results correlate with an independent measure of skill.

7.4 Building order inside each phase

1. Build the smallest version that can be used in a real session.
2. Use it with real students within two weeks.
3. Measure, fix the worst problem, and repeat.

Polish comes after evidence, not before.

CHAPTER 8

Making VANTA successful in industry

8.1 Four markets VANTA can serve

Market	The customer's pain	What VANTA offers	Who pays
Universities and courses	Assignments are reused year to year; solutions circulate; plagiarism checks are after the fact; AI assistance blurs authorship	A unique world per student for the same learning goal; autograding; debriefs; an instructor console	Departments (per course or seat licence); later, institutions
Competitions and communities	Problem-setting is expensive and leaks; events are hard to grade fairly at scale	A generated, proven problem set per event; an automatic core score; formats from Sprint to Arena	Sponsors, event platforms, colleges' tech fests
Hiring and placements	Question banks leak; AI-assisted cheating; puzzle questions don't resemble the job	"Fix this real system" assessments; uniqueness per candidate; process evidence; trade-off reasoning	Companies (per candidate or per seat); placement cells
Corporate training	Onboarding to unfamiliar codebases; incident-response practice; upskilling	Incident drills on generated systems; debugging and performance tracks	Engineering teams, L&D budgets

Market size, with care. Market-research estimates for assessment software vary widely between publishers. As one data point, Verified Market Research puts *technical skills screening software* at about US$340 million in 2024, growing to about US$581 million by 2031 (around 7% a year). Its named players include HackerRank, Codility, HackerEarth and DevSkiller. Treat such figures as rough orders of magnitude, not facts to build a plan on.

8.2 Where VANTA sits

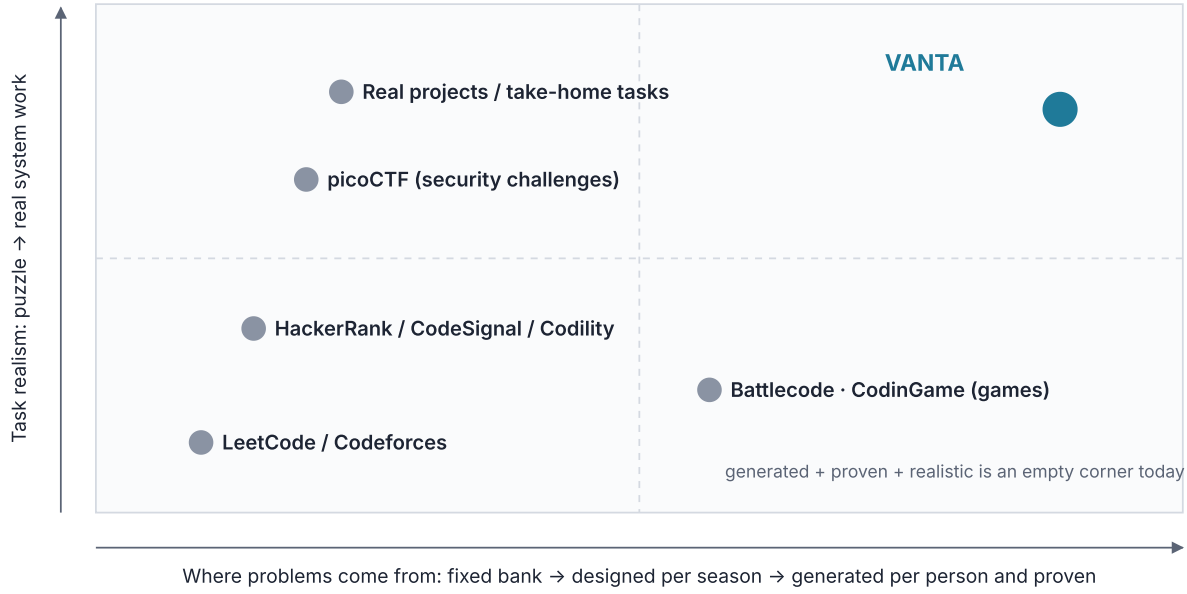


Figure 8.1. A qualitative positioning map (positions are judgement, not measurement). VANTA's opening is the top-right: realistic work, with problems that are generated per person and proven.

8.3 The competitive landscape

Platform	What it does well	Where VANTA differs
LeetCode, Codeforces, AtCoder	Huge problem sets, strong communities, contests and ratings	Static, hand-written problems; blank-page algorithm puzzles. VANTA gives realistic broken systems, unique per person
HackerRank, CodeSignal, Codility, HackerEarth	Hiring assessments, proctoring, plagiarism and leak detection, ATS integrations	They fight leaks after the fact: HackerRank scans the web daily and replaces leaked questions. VANTA avoids leaks by generating every world
CodinGame	Bot-programming arenas with leagues, boss bots and TrueSkill ranking	Hand-designed games. VANTA Arena adds generated maps and seasons, and planted bugs in the baseline bot
Battlecode (MIT)	A legendary annual strategy contest with sprint, qualifier and final tournaments	One game a year, run by MIT. VANTA could offer the format to any college, any time
picoCTF	Competition as a learning on-ramp; 600,000+ users; a practice gym after contests	Security-focused. VANTA does the same for software engineering skills
Gradescope and other autograders	Course logistics and autograding, similarity checks	Grade whatever assignment you write. VANTA generates the assignment and proves it

The honest conclusion: incumbents are strong at **distribution** (customers, integrations, brand). VANTA is strong at **content generation and realism**. The realistic path is to win a niche where generation matters most (courses and competitions), build evidence, and then either grow into assessments or partner with a platform that already has distribution.

8.4 Business models that fit

Model	How it works	Fits phase
Free for students and clubs	Practice, leagues and small contests stay free	Phases 1–3 (growth and data)
Course licence	Per course per semester: unique assignments, instructor console, debriefs	Phases 2–3
Sponsored seasons	A company sponsors an Arena season or a Sprint series for visibility with strong students	Phases 2–3
Event-platform partnership	VANTA as the "generated problems" engine inside existing hackathon and contest platforms	Phase 3
Per-candidate assessments	Companies pay per assessed candidate; placement cells pay per drive	Phase 3+, after validity evidence
Open core	The engine and archetype format are open source; hosted contests, the instructor console and assessments are paid	An option to decide in Phase 3

Sponsored seasons have a natural opening. Google's competitions engaged more than 300,000 engineers a year and helped Google hire thousands. They were closed in February 2023 as a cost cut. Companies still want access to strong students. A low-cost, generated, fair format lowers the cost of providing it.

8.5 Going to market: start where you are

1. **Your university.** The pilot study and Sprint #1 (Chapters 3 and 6). A faculty sponsor and one course section.
2. **Student clubs and fests.** Coding clubs, ACM/IEEE chapters, tech fests: a Sprint or Incident Night as an event they can run themselves.
3. **Nearby colleges.** An inter-college Sprint, then an Arena season. Bengaluru has a dense college and tech-company ecosystem within reach.
4. **Instructors.** Offer a free semester of course use in exchange for data and a testimonial.
5. **Companies.** Once there is data, approach companies that hire from those colleges with a sponsored season or a placement-assessment pilot.

India context: the Mercer-Mettl India Graduate Skill Index 2025 reports 42.6% overall employability among graduates it assessed (data from over 2,700 campuses). A tool that builds *and* demonstrates practical skill speaks directly to that gap, for students, colleges and employers alike.

8.6 What makes it defensible

- **The archetype library and generator.** Each archetype is engineering work: realistic framing, variants, a harness that really triggers the bug, and two distinct fixes. A large, tested library is hard to copy quickly.
- **The proof loop.** "Served only if proven broken, fixable, and fixable in two ways" is a quality guarantee that static problem banks can't make.
- **Behavioural data.** How people investigate (which experiments, which witnesses, in what order) is unusual data. It can improve hints, difficulty calibration and, eventually, assessment validity.
- **Community and seasons.** Annual Arena seasons create a tradition, as Battlecode has shown since 2007.

8.7 Metrics that matter

Area	Metric	Good early target
Learning	Post-test solve-rate gain on fresh seeds	A clear, repeatable improvement over the pre-test
Engagement	Weekly active solvers; 30-day return rate	A growing base; students coming back without being required to
Competitions	Participants per event; completion rate; would-return %	≥ 70% would return
Content	Archetypes live; viable-world rate; generation time	+1–2 archetypes a month; < 30 s generation
Operations	Grading p95 latency at peak; sandbox incidents	< 5 s; zero incidents
Assessment (later)	Correlation with an independent skill measure; fairness across groups	Evidence before selling

8.8 What a professor, a company, or an investor will want to see

1. A clear one-sentence scope (Chapter 1) and a working demo.
2. **Data from real students**: the pilot results, Sprint #1 numbers, survey quotes.
3. Breadth: 6–10 archetypes, covering debugging, DSA and performance.
4. Safety: a containerized sandbox, an AI policy, integrity features.
5. A partner: a course using it, a club running events, or a sponsor.
6. A plan for continuity after you graduate: a club, the department, or open source.

CHAPTER 9

Risks and how to handle them

Risk	Why it matters	Mitigation
AI solves the worlds	Undermines contests and assessments if rules aren't clear	Tiered AI rules; unique worlds; process scoring; formats where sustained judgement wins (Arena, Marathon, Hackathon); live "defend your fix" for finals. Never claim worlds are AI-proof
Sandbox escape or abuse	Running untrusted C on a server is risky	Containers with no network, read-only filesystems, CPU, memory and process limits; separate worker machines; keep the static checks as an extra layer
Content bottleneck	Two archetypes are not enough for a season of contests	An archetype authoring kit; one or two new archetypes a month; a review process for contributions; LLM help to *draft* archetypes, always verified by the evolution loop
Unfair timing in performance worlds	Machine noise could decide results	Time limits derived from reference runs with generous margins; the median of repeats; operation counters for Arena
Generated worlds feel repetitive	Students notice templates	More settings and variants per archetype; multi-subsystem worlds; seasonal games
Cold start	Contests need participants; courses need instructors	Start at your own university with a faculty sponsor; partner with clubs; keep student use free
Founder bandwidth	You are a first-year student with coursework	One mode at a time; recruit 2–3 co-builders from the first event; scope cuts (Chapter 2)
Incumbents copy the idea	Big platforms have distribution	Move fast on the niche (courses and competitions), build the library and community, consider partnering instead of competing
Assessment validity and fairness	Hiring use has legal and ethical weight	Don't sell assessments until there is validity and fairness evidence; start with practice and placements that don't decide outcomes alone
Continuity after you graduate	Student projects often die with the founder	Hand over to a club or the department; open-source the engine; document everything (the technical PDF already exists)

CHAPTER 10

Your next steps

Concrete actions for the coming weeks, in order. Each one produces something you can show.

Action	Output	Rough effort
1. Adopt the scope sentence and the three-mode framing (Learn, Compete, Assess) in your pitch	A clear story for your professor	1 hour
2. Add the **debrief** screen (the planted bug in words, the evidence trail, other approaches)	VANTA teaches, not just tests	3–4 days
3. Write the first **DSA archetype**: binary search in a seat finder, 3 variants, 2 distinct fixes	DSA enters VANTA	1 week
4. Write the first **too-slow archetype**: duplicate session finder with measured time limits	Competitive programming enters VANTA	1 week
5. Build **contest mode** + **scoreboard** (pinned problems, timer, ICPC penalties, freeze)	VANTA can host a Sprint	1–1.5 weeks
6. **Containerize the sandbox** and add a worker queue	Safe for a public event	3–5 days
7. Run the **pilot study** (20–40 students, pre-test → 3 sessions → post-test)	Real learning data	4–5 weeks, alongside the other work
8. Run **Sprint #1** (Chapter 6)	An event, plus numbers and quotes	1 evening, plus preparation
9. Write a **2-page results report** and present it with a demo	The pitch that gets support	2 days
10. Recruit 2–3 co-builders from the participants; plan **Arena season 1** for next semester	A team and a flagship	Ongoing

10.1 How to present this to your professor

1. **The problem.** Fixed problem banks leak and get solved by AI. Puzzle questions don't teach real engineering. Debugging, the most common real task, is rarely taught explicitly.
2. **The idea.** An engine that generates realistic broken systems, proves them by running them, and supports many correct answers. The scope map (Figure 2.1).
3. **The demo.** Generate, investigate, fix, submit, rating (5 minutes, using the technical document's demo script).
4. **The plan.** Learn → Compete → Assess (Chapter 1). Phase 1 deliverables (Chapter 7).
5. **The ask.** A course section or lab for the pilot; a room and a date for Sprint #1; faculty sponsorship for an inter-college event next semester.

Remember what makes this credible

Not the number of features, but three things: it works live, real students used it and improved, and there is a clear next step that the professor can say yes to.

APPENDIX A

Proposed archetype catalogue

Twenty proposed defect classes, beyond the two that exist today. The "Kind" column refers to Chapter 4: **A** broken data structure or logic, **B** too slow, **C** optimization, **D** adversarial game, **M** a multi-subsystem hackathon world. Every row needs at least two distinct reference fixes before it can be served.

A.1 Catalogue

System (story)	Kind	Topic	Planted variants (examples)	Distinct correct fixes	Track
1. Seat finder for exam halls	A	Binary search	`lo <= hi` loop that never ends; returns any match instead of the first; mid-point overflow	Closed-interval lower bound; half-open interval	DSA in the Wild
2. Hospital triage queue	A	Binary heap	Sift-down compares with the wrong child; size not decremented on pop; parent index off by one	Array heap; bucket queue for small priority ranges	DSA in the Wild
3. Username registry	A	Hash table	Deletion breaks probe chains; no resize at high load; negative hash with `%`	Tombstones; backward-shift deletion; separate chaining	DSA in the Wild
4. Course prerequisite checker	A	Graphs, topological sort	Visited marked on pop; cycle check only from node 0; in-degree not decremented	Kahn's algorithm; DFS with three colours	DSA in the Wild
5. Network outage regions	A	Union-find	Union attaches the wrong root; `find` drops the compressed root	Union by rank; union by size with path halving	DSA in the Wild
6. Session cache	A	LRU (hash + linked list)	Eviction removes the most recent item; stale pointer after a move-to-front; capacity check off by one	Hash + doubly linked list; hash + timestamps with lazy eviction	DSA in the Wild
7. Message relay buffer	A	Ring buffer	Full/empty confusion; wrap-around index bug; overwrite without advancing the tail	Count field; one-empty-slot convention	C Foundations
8. Exam marks aggregator	A	Integers, overflow	Integer-division average; `int` overflow on totals; rounding toward zero for negatives	64-bit accumulation; scaled integer arithmetic	C Foundations
9. Log line parser	A	Strings, memory	Missing terminator; off-by-one buffer copy; reading past the line end	Length-bounded parsing; tokenizer with explicit lengths	C Foundations

System (story)	Kind	Topic	Planted variants (examples)	Distinct correct fixes	Track
10. Duplicate session finder	B	Hashing, sorting	$O(n^2)$ pair comparison	Hash set; sort and scan	Performance
11. Top-k trending tags	B	Heaps, selection	Full re-sort for every query	Size-k min-heap; counting with bucket selection	Performance
12. Room booking conflict checker	B	Intervals, sweep	Scan every booking per query	Sort + sweep; difference array for bounded times	Performance
13. Campus route planner	B	Shortest paths	BFS on weighted edges; Dijkstra with a linear-scan minimum	Dijkstra with a binary heap; 0-1 BFS where weights allow	Performance
14. Sensor rolling average	B	Prefix sums, windows	Recompute the window sum every tick	Running sum; prefix sums	Performance
15. Friend-group counter	B	Incremental connectivity	Flood fill after each new friendship	Union-find; merge-smaller-into-larger sets	Performance
16. Course search autocomplete	B	Tries, sorted search	Linear scan of all names per keystroke	Trie; sorted array + binary search for the prefix range	Performance
17. Exam hall scheduler	C	Scheduling, local search	Baseline: first-fit in input order	Greedy by constraint degree + local search; simulated annealing	Systems & Trade-offs
18. Delivery batching	C	Bin packing, routing	Baseline: one delivery per trip	First-fit decreasing; savings-style route merging	Systems & Trade-offs
19. "Contention" arena game	D	Pathfinding, scheduling, budgets	Baseline bot with planted bugs (Appendix C)	Many strategies by design	Arena
20. Campus facilities platform	M	Integration of several of the above	4–8 planted defects across booking, laundry and lockers, plus conflicting feature requests	Per subsystem, as above	Hackathon

A.2 What each archetype must include

Taken from the technical documentation's extension checklist (Chapter 12 there):

- 3 or more story settings, wide identifier pools, and 3 or more variants;
- a harness whose hidden tests really trigger each variant, and check *all* requirements in every test;
- 2 or more architecturally distinct reference fixes;
- `params` for the probe lab and witnesses, plus a probe kit and a witness kit;
- tests: viable for every pinned variant; starter rejected and fix accepted under sanitizers; no hints in the starter;
- for kind B: measured time limits, with the starter timing out and both fixes passing with margin.

APPENDIX B

Scoring formulas

B.1 Sprint (ICPC rules)

Rank teams by the number of worlds solved (more is better), then by total penalty time (less is better):

```
penalty = Σ over solved worlds ( minutes_until_accepted + 20 × rejected_runs_before_acceptance )
```

Rejected runs on worlds that were never solved add nothing. Optionally, compile errors don't count as rejected runs. Tie-break by the earlier time of the last accepted run.

Optional VANTA variant (decaying points). Each world has a value V (for example 500 for A up to 3000 for F) that decays with time t (minutes) and wrong attempts w:

```
score(world) = max( 0.3 × V,  V × (1 − t / (2 × contest_minutes)) − 50 × w )
```

This rewards speed on easy worlds without making late solves worthless.

B.2 Marathon (relative scoring)

For each hidden instance i, compare your result r_i with the best result submitted by anyone, b_i:

```
maximization:  sᵢ = 10⁹ × rᵢ / bᵢ          minimization:  sᵢ = 10⁹ × bᵢ / rᵢ
total = Σ sᵢ          (an invalid output on an instance scores 0 for that instance)
```

Show provisional standings on public instances during the contest. Compute final standings on a separate hidden set, as AtCoder Heuristic Contest does with its system tests.

B.3 Incident Night (dynamic points)

Each world starts at *initial* points and loses value as more teams solve it, down to a floor. With s = number of solves so far and d = the number of solves at which the floor is reached:

```
value(s) = max( floor,  initial − (initial − floor) × (s / d)² )
first-solve bonus = 10% of initial (awarded once)
```

Example: initial 500, floor 100, d = 20. Worth 500 at 0 solves, 400 after 10, and 100 from 20 solves on. Every team that solved the world gets its *current* value, so the scoreboard updates as others solve.

B.4 Arena (ratings and matches)

- **A match** is played on a seeded map. The winner has the higher score: delivered value minus lateness penalties (see Appendix C). The same seed and the same bots always give the same result.
- **Ladder rating.** Use Elo for simplicity. With rating R, opponent rating O, K = 24 and S = 1 for a win, 0.5 for a draw and 0 for a loss:

```
expected = 1 / (1 + 10^((O − R) / 400));     R ← R + K × (S − expected)
```

TrueSkill (as used by CodinGame) handles multi-player matches and uncertainty better, and is a good upgrade when matches have 3–4 teams.

- **Leagues.** Boss bots gate promotion: beat the league's boss bot in at least 60% of games over a fixed map set. No demotion within a season.
- **Tournaments.** The Sprint tournament is single elimination, with each pairing played over 5 seeded maps. Qualifiers and finals are double elimination, with each pairing played over 7 maps.

B.5 Hackathon (rubric)

Component	Weight	Formula
Correctness	50	50 × (hidden checks passed ÷ total checks)
Stakeholder satisfaction	20	20 × mean over witnesses of (their satisfaction under your code), each scaled 0–1 from their trace (e.g. waits and refusals relative to the reference fixes)
Performance	10	10 × (large hidden tests within the limit ÷ total)
Engineering quality	20	Judges' rubric: clarity of explanation (8), trade-off awareness (6), code quality (6)

B.6 Practice points (Learn mode)

Event	Points
Solve a world	100 × (1 + difficulty)
Hint rung 1 / 2 / 3 / 4	−10 / −20 / −30 / −40
Solve without a rejected run	+20
Explanation peer-rated as clear	+20
Spaced-review world solved on time	+30

Practice points are for motivation only. The rating (Elo-style, already built) remains the measure of skill used for calibration and divisions.

APPENDIX C

Arena game sketch, "Contention"

A first Battlecode-style game for VANTA Arena, designed around the defect classes VANTA already knows: shared-resource scheduling, pools that can leak, and fairness.

C.1 The world

- **A generated grid city** of 40×40 to 64×64 cells. The seed decides the layout. It contains:
- roads, plus narrow one-lane bridges that only one courier can occupy;
- depots where requests appear;
- customers;
- charging docks, a shared pool with limited slots.
- **Each team** (2–4 per match) controls 4–8 couriers. A courier carries one package and has a battery.
- **Requests** appear over time at depots. Each has a destination, a value, and a deadline. Late deliveries lose value, and very late ones lose all of it.
- **Docks** recharge batteries. A courier must *claim* a dock slot and *release* it when done. Slots that are claimed and never released are lost for the rest of the match, like a pool leak.

C.2 A turn

```
struct View   { /* your couriers, visible cells (radius 6), visible requests, dock states, turn */ };
struct Orders { /* per courier: MOVE dir | PICKUP | DROP | CLAIM_DOCK | RELEASE_DOCK | WAIT */ };

void act(const struct View *view, struct Orders *out);   /* one call per team per turn */
```

- The game runs a fixed number of turns (for example 1,500). It is fully deterministic given the map seed and the bots' code.
- **Compute budget.** Each `act` call may perform a fixed number of counted operations (for example 200,000 per turn, with a small bank for unused budget). Exceeding it ends the call, and unfinished orders become WAIT. This rewards efficient algorithms, as Battlecode's computation tracking does.
- **Fog of war.** A team sees only what its couriers can see. Shared knowledge must be communicated through a small team "radio" array, which adds a data-structure design question.

C.3 Scoring a match

```
score = Σ delivered value  −  Σ lateness penalties  −  5 × (dock slots leaked by your team)
```

Ties are broken by the earlier time of the last delivery, then by the number of deliveries.

C.4 The baseline bot and its planted bugs

Every team receives the same starter bot. It plays, but badly, because it contains VANTA-style defects. Finding them is week 1's challenge:

Planted defect	Symptom in matches	What fixing it teaches
BFS pathfinder marks cells visited on pop, not push	Couriers stall when the map is crowded; budget runs out	BFS correctness; complexity under budget
Dispatcher always takes the nearest request (pure greedy)	High-value distant requests expire; witnesses report long waits	Fairness vs. throughput; priority with aging
Dock claim not released when a courier's battery fills during a wait	Docks leak; the team runs out of charging	Resource pool integrity
Radio array index computed with the wrong modulus	Teammates overwrite each other's notes	Off-by-one and ring-buffer logic

Arena witnesses. After each scrimmage, the match trace drives witness answers. Examples: a customer ("my parcel arrived 40 turns late"), a dock operator ("slot 3 was claimed by your courier at turn 210 and never released"), and a rival ("your couriers blocked the north bridge for 90 turns"). Investigating your own matches is part of the strategy.

C.5 The DSA a strong bot needs

- **BFS / A*** for routing; multi-source BFS for distance maps.
- **Priority queues** for scheduling deliveries by value, deadline and distance.
- **Assignment** (greedy with repair, or Hungarian on small sets) to match couriers to requests.
- **Caching** of distance maps under the compute budget.
- **Ring buffers and compact encodings** for the team radio.

C.6 A suggested season calendar (4 weeks)

Week	Event
0	Release: rules, starter bot (with planted bugs), the replay viewer, a practice map pool
1	Scrimmage ladder opens; end of week: **Sprint tournament** (single elimination)
2	Rule tweak announced (e.g. a new building type) to reward adaptation, as Battlecode does
3	**Qualifier tournament** (double elimination); top 8 advance
4	**Live final** with replays on a big screen; winners present their strategies; post-season practice ladder stays open

APPENDIX D

Sources

Facts in this document come from the sources below, checked in October 2026. Market-size figures come from a commercial market-research publisher; such estimates vary widely and should be treated as rough. Statistics on AI-assisted cheating come from industry write-ups and are quoted cautiously in the text.

D.1 Competitions and formats

Topic	Source
Battlecode format, languages, tournaments, prize pool	battlecode.org — About
Battlecode history (since 2007) and skills	MIT EECS — "Robot armies duke it out in Battlecode's epic on-screen battles"
ICPC ranking and the 20-minute penalty rule	CERC 2025 — General rules
AtCoder Heuristic Contest format	AtCoder Heuristic Contest 025
CodinGame leagues, boss bots, TrueSkill	CodinGame blog — bot programming leagues
Google closing Code Jam, Hash Code, Kick Start	The Pragmatic Engineer — "Why did Google close its coding competitions after 20 years?"
Smart India Hackathon 2025 numbers	News On AIR — 8th edition of Smart India Hackathon 2025 begins
picoCTF reach and picoGym	Carnegie Mellon University — picoCTF accessibility

D.2 AI and integrity

Topic	Source
Gemini at ICPC World Finals 2025	Google DeepMind — Gemini achieves gold-medal level at ICPC World Finals
Codeforces rule restricting AI use (2024-09-14)	Codeforces — Rule restricting the use of AI
HackerRank leaked-question detection	HackerRank Knowledge Base — Manage leaked questions
HackerRank AI-use policies and monitoring	HackerRank — How recruiters see candidates' AI usage during tests
Industry claims about AI interview cheating	Metamindz — Karat vs HackerRank vs CodeSignal on AI interview cheating

D.3 Learning science

Topic	Source
Productive failure meta-analysis (Sinha & Kapur, 2021)	World Economic Forum summary; original: *Review of Educational Research*, doi:10.3102/00346543211019105
Debugging in CS education	McCauley, R. et al. (2008). Debugging: a review of the literature from an educational perspective. *Computer Science Education*, 18(2), 67–92. Record

D.4 Markets and employability

Topic	Source
Technical skills screening software market estimate	Verified Market Research — Technical Skills Screening Software Market
India graduate employability (42.6%)	Mercer-Mettl — India Graduate Skill Index 2025

D.5 VANTA itself

Topic	Source
Everything about the current system	`docs/VANTA-Technical-Documentation.pdf` in the repository